`std::multi_lock`

- **Document number:** P3833R1
- **Date:** 2026-03-24
- **Audience:** LEWGI/SG1
- **Project:** ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

- **Reply-to:** Ted Lyngmo ted@lyncon.se

Revision History

R1

- Rewrote postconditions throughout the wording to use prose ("is equal to", "is `true`", "is `false`") instead of `==` notation.

Abstract

This paper proposes `std::multi_lock`, a RAII class template that combines the functionality of `std::unique_lock` and `std::scoped_lock`. Unlike `std::scoped_lock`, which provides only basic RAII semantics, `std::multi_lock` offers the full flexibility of `std::unique_lock` (deferred locking, try-lock operations, timed locking, and ownership transfer) while supporting multiple mutexes simultaneously.

Contents

2. Motivation

The C++ standard library provides several mutex wrapper classes:

- `std::lock_guard`: Simple RAII wrapper for a single mutex
- `std::unique_lock`: Flexible RAII wrapper for a single mutex with deferred locking, try-lock, and ownership transfer
- `std::scoped_lock`: RAII wrapper for multiple mutexes with deadlock avoidance

However, there is no facility that combines the flexibility of `std::unique_lock` with the multi-mutex capabilities of `std::scoped_lock`. While `std::scoped_lock` provides excellent RAII semantics for multiple mutexes, it always locks immediately at construction and cannot be used in situations requiring:

- **Deferred locking**: Constructing the lock object before deciding whether to lock
- **Timed locking**: Attempting to acquire locks with a timeout
- **Try-lock operations**: Non-blocking lock attempts with graceful failure handling

These are common patterns when working with a single mutex using `std::unique_lock`, but developers currently have no equivalent option for multiple mutexes. This gap forces developers to

choose between:

1. Using `std::scoped_lock` and restructuring code to avoid deferred/timed locking scenarios
2. Managing multiple `std::unique_lock` objects manually, which is verbose and error-prone

The following table illustrates how `multi_lock` completes the family of mutex wrappers:

	One mutex	**Zero or more mutexes**
Always owning	`lock_guard`	`scoped_lock`
Flexible (*)	`unique_lock`	`multi_lock`

(*) Supports deferred locking, time-constrained attempts at locking, recursive locking, and transfer of lock ownership.

3. Proposed Solution

This paper proposes `std::multi_lock<Ms...>`, a variadic class template that:

- Accepts zero or more mutex types satisfying *Cpp17BasicLockable*, *Cpp17Lockable*, or *Cpp17TimedLockable* requirements
- Provides RAII semantics with automatic unlocking
- Supports deferred locking, try-lock, and adopt-lock strategies
- Supports timed locking operations when all mutexes are *Cpp17TimedLockable*
- Uses deadlock-free acquisition of multiple mutexes
- Allows ownership transfer via move semantics. While this isn't a primary use-case it makes sense to provide the same functionality as `unique_lock` in this regard.

Use Cases

Example 1: Deferred locking of multiple mutexes

```
std::mutex m1, m2;
std::multi_lock lock(std::defer_lock, m1, m2);
// ... prepare work ...
if (condition) {
    lock.lock(); // Deadlock-safe locking
    // critical section
}
```

Example 2: Timed locking with timeout

```
std::multi_lock lock(100ms, m1, m2);
if (lock) {
    // Successfully acquired all locks within timeout
    // critical section
}
```

Example 3: Conditional locking with manual control

```
std::multi_lock lock(std::try_to_lock, m1, m2);
if (!lock) {
    // Could not acquire all locks, handle gracefully
    return;
}
```

Example 4: Replacing a `scoped_lock` with deferred and timed locking

Before:

```
std::mutex m1, m2;
//...
std::scoped_lock lock(m1, m2);
// critical section
```

After:

```
std::timed_mutex m1, m2;
//...
```

```
std::multi_lock lock(std::defer_lock, m1, m2);
for(int rv; (rv = lock.try_lock_for(100ms)) != -1;) {
    // log something or take some other action
}
// critical section
```

4. Impact on the Standard

This proposal is a pure library extension. It adds a new class template `std::multi_lock` to the `<mutex>` header and updates related sections of the standard to reference it. There are no changes to the core language and no breaking changes to existing code.

5. Design Decisions

Return value for try-lock operations

Following `std::try_lock`, the `try_lock()`, `try_lock_for()`, and `try_lock_until()` member functions return `int`: - `-1` on success (all locks acquired) - Otherwise, the 0-based index of the mutex that failed to lock

This maintains consistency with existing standard library facilities.

Timed locking with multiple mutexes

The standard library currently lacks `std::try_lock_for` and `std::try_lock_until` functions for multiple mutexes. P3832 proposes to add these free functions. This proposal includes corresponding `std::multi_lock` member functions that provide this functionality, possibly by using the two proposed free functions.

Exception Safety

`std::multi_lock` provides the following exception safety guarantees:

- **Basic guarantee**: If any operation throws, the object remains in a valid state. All successfully acquired locks are released.

- **Strong guarantee**: Failed lock attempts (via `try_lock`, `try_lock_for`, or `try_lock_until`) leave all mutexes unlocked. If any mutex fails to lock, all previously acquired locks in that operation are released before returning.
- **No-throw guarantee**: Move operations (`multi_lock(multi_lock&&)` and `operator=(multi_lock&&)`), `swap()`, and `release()` never throw exceptions.

The destructor provides the basic guarantee: if `owns_lock()` is `true`, it calls `unlock()`, which may throw but ensures all mutexes are unlocked before propagating the exception.

Deadlock Avoidance

When locking multiple mutexes, `std::multi_lock` must avoid deadlock. The implementation can use a similar deadlock-avoidance algorithm as `std::lock()` for multiple mutexes:

- For `lock()`: It can use `std::lock(*get<Is>(pm)...)` which locks all mutexes via a sequence of calls that does not result in deadlock, but is otherwise unspecified.
- For `try_lock()`: Attempts to lock mutexes in order and backs off if any lock fails. It can use `std::try_lock(*get<Is>(pm)...)` for this purpose.
- For `try_lock_until()` and `try_lock_for()`: It can use an algorithm similar to `std::lock()` but with timed operations, as proposed in P3832.

6. Technical Specification

Header `<mutex>` synopsis

```
namespace std {
  template<class... MutexTypes>
  class multi_lock;

  template<class... MutexTypes>
  void swap(multi_lock<MutexTypes...>& x, multi_lock<MutexTypes...>& y)
        noexcept;
}
```

Class template `multi_lock`

```
namespace std {
  template<class... MutexTypes>
  class multi_lock {
  public:
    using mutex_type = tuple<MutexTypes*...>;

    // Constructors
    multi_lock() noexcept;
    explicit multi_lock(MutexTypes&... m);
    multi_lock(defer_lock_t, MutexTypes&... m) noexcept;
    multi_lock(try_to_lock_t, MutexTypes&... m);
    multi_lock(adopt_lock_t, MutexTypes&... m) noexcept;

    template<class Rep, class Period>
    multi_lock(const chrono::duration<Rep, Period>& timeout_duration,
        MutexTypes&... m);

    template<class Clock, class Duration>
    multi_lock(const chrono::time_point<Clock, Duration>& timeout_time,
        MutexTypes&... m);

    // Destructor
    ~multi_lock();

    // Move operations
    multi_lock(multi_lock&& other) noexcept;
    multi_lock& operator=(multi_lock&& other) noexcept;

    // Deleted copy operations
    multi_lock(const multi_lock&) = delete;
    multi_lock& operator=(const multi_lock&) = delete;

    // Locking operations
    void lock();
    int try_lock();

    template<class Rep, class Period>
    int try_lock_for(const chrono::duration<Rep, Period>&
        timeout_duration);
```

```
    template<class Clock, class Duration>
    int try_lock_until(const chrono::time_point<Clock, Duration>&
        timeout_time);

    void unlock();

    // Modifiers
    void swap(multi_lock& other) noexcept;
    mutex_type release() noexcept;

    // Observers
    mutex_type mutex() const noexcept;
    bool owns_lock() const noexcept;
    explicit operator bool() const noexcept;

  private:
    mutex_type pm; // exposition only
    bool owns;     // exposition only
  };

  template<class... MutexTypes>
  void swap(multi_lock<MutexTypes...>& x, multi_lock<MutexTypes...>& y)
       noexcept;
}
```

7. Alternative Designs Considered

Container-based Interface

An alternative design would use a container-based interface:

```
multi_lock(std::vector<std::mutex*> mutexes);
```

Or with `std::span`:

```
template<class Mutex>
multi_lock(std::span<Mutex*> mutexes);
```

Rejected: Both approaches require all mutexes to be of the same type. While `std::span` avoids ownership concerns and could be non-owning, it still cannot mix different mutex types (e.g., `std::mutex` and `std::timed_mutex` in the same lock). The variadic template approach maintains type safety, allows heterogeneous mutex types, and enables compile-time optimizations that would not be possible with a runtime container. Additionally, it is probably preferable to keep the interface similar to that of `unique_lock` and `scoped_lock` to complete the family of mutex wrapper classes with a consistent design.

8. Implementation Experience

A complete implementation is available at github.com/bemanproject/timed_lock_alg. The implementation has been tested with multiple mutex types and demonstrates that the design is implementable and practical.

9. Wording

The following changes are relative to N5014.

32.2.5.1 General [thread.req.lockable.general]

Modify paragraph 3:

The standard library templates `unique_lock` (32.6.5.4), `shared_lock` (32.6.5.5), `multi_lock` (32.6.X), `scoped_lock` (32.6.5.3), `lock_guard` (32.6.5.2), `lock`, `try_lock` (32.6.6), and `condition_variable_any` (32.7.5) all operate on user-supplied lockable objects. The *Cpp17BasicLockable* requirements, the *Cpp17Lockable* requirements, the *Cpp17TimedLockable* requirements, the *Cpp17SharedLockable* requirements, and the *Cpp17SharedTimedLockable* requirements list the requirements imposed by these library types in order to acquire or release ownership of a lock by a given execution agent.

32.6 Mutual exclusion [thread.mutex]

32.6.2 Header `<mutex>` synopsis [thread.mutex.syn]

Add to the header synopsis after the declaration of class template `scoped_lock`:

```
// 32.6.X, class template multi_lock
template<class... MutexTypes>
  class multi_lock;

template<class... MutexTypes>
  void swap(multi_lock<MutexTypes...>& x, multi_lock<MutexTypes...>& y)
       noexcept;
```

32.6.X Class template `multi_lock` [thread.lock.multi]

32.6.X.1 General [thread.lock.multi.general]

```
namespace std {
  template<class... MutexTypes>
  class multi_lock {
  public:
    using mutex_type = tuple<MutexTypes*...>;

    // 32.6.X.2, construct/copy/destroy
    multi_lock() noexcept;
    explicit multi_lock(MutexTypes&... m);
    multi_lock(defer_lock_t, MutexTypes&... m) noexcept;
    multi_lock(try_to_lock_t, MutexTypes&... m);
    multi_lock(adopt_lock_t, MutexTypes&... m) noexcept;
    template<class Rep, class Period>
    multi_lock(const chrono::duration<Rep, Period>& rel_time,
        MutexTypes&... m);
    template<class Clock, class Duration>
    multi_lock(const chrono::time_point<Clock, Duration>& abs_time,
        MutexTypes&... m);
    ~multi_lock();

    multi_lock(const multi_lock&) = delete;
    multi_lock& operator=(const multi_lock&) = delete;

    multi_lock(multi_lock&& u) noexcept;
    multi_lock& operator=(multi_lock&& u) noexcept;

    // 32.6.X.3, locking
```

```
    void lock();
    int try_lock();
    template<class Rep, class Period>
    int try_lock_for(const chrono::duration<Rep, Period>& rel_time);
    template<class Clock, class Duration>
    int try_lock_until(const chrono::time_point<Clock, Duration>&
        abs_time);
    void unlock();

    // 32.6.X.4, modifiers
    void swap(multi_lock& u) noexcept;
    mutex_type release() noexcept;

    // 32.6.X.5, observers
    bool owns_lock() const noexcept;
    explicit operator bool() const noexcept;
    mutex_type mutex() const noexcept;

  private:
    mutex_type pm;  // exposition only
    bool owns;      // exposition only
  };

  template<class... MutexTypes>
  void swap(multi_lock<MutexTypes...>& x, multi_lock<MutexTypes...>& y)
       noexcept;
}
```

1 A `multi_lock` controls the ownership of lockable objects within a scope. Ownership of the lockable objects may be acquired at construction or after construction, and may be transferred, after acquisition, to another `multi_lock` object. `multi_lock` specializations are not copyable but are movable. The behavior of a program is undefined if any contained pointer in `pm` is not null and the lockable object pointed to does not exist for the entire remaining lifetime (6.8.4) of the `multi_lock` object. All types in the template parameter pack `MutexTypes` shall meet the *Cpp17BasicLockable* requirements (32.2.5.2).

2 [*Note 1*: `multi_lock<MutexTypes...>` meets the *Cpp17BasicLockable* requirements. If all types in `MutexTypes` meet the *Cpp17Lockable* requirements (32.2.5.3), `multi_lock<MutexTypes...>` also meets the *Cpp17Lockable* requirements; if all types in

`MutexTypes` meet the *Cpp17TimedLockable* requirements (32.2.5.4), `multi_lock<MutexTypes...>` also meets the *Cpp17TimedLockable* requirements. — *end note*]

32.6.X.2 Constructors, destructor, and assignment [thread.lock.multi.cons]

```
multi_lock() noexcept;
```

1 *Postconditions*: `pm` is equal to `tuple<MutexTypes*...>{}` and `owns` is `false`.

```
explicit multi_lock(MutexTypes&... m);
```

2 *Constraints*: `sizeof...(MutexTypes) > 0` is `true`.

3 *Preconditions*: If `sizeof...(MutexTypes)` does not equal 1, all types in `MutexTypes` meet the *Cpp17Lockable* requirements (32.2.5.3).

4 *Effects*: Initializes `pm` with `addressof(m)...`. Then calls `lock()`.

5 *Postconditions*: `pm` is equal to `tuple<MutexTypes*...>{addressof(m)...}` and `owns` is `true`.

```
multi_lock(defer_lock_t, MutexTypes&... m) noexcept;
```

6 *Effects*: Initializes `pm` with `addressof(m)...`.

7 *Postconditions*: `pm` is equal to `tuple<MutexTypes*...>{addressof(m)...}` and `owns` is `false`.

```
multi_lock(try_to_lock_t, MutexTypes&... m);
```

8 *Preconditions*: All types in `MutexTypes` meet the *Cpp17Lockable* requirements (32.2.5.3).

9 *Effects*: Initializes `pm` with `addressof(m)...`. Then calls `try_lock()`.

10 *Postconditions*: `pm` is equal to `tuple<MutexTypes*...>{addressof(m)...}` and `owns` is `true` if `res` equals -1, otherwise `false`, where `res` is the value returned by `try_lock()`.

```
multi_lock(adopt_lock_t, MutexTypes&... m) noexcept;
```

11 *Preconditions*: The calling thread holds a non-shared lock on each element of `m`.

12 *Effects*: Initializes `pm` with `addressof(m)...`.

13 *Postconditions*: `pm` is equal to `tuple<MutexTypes*...>{addressof(m)...}` and `owns` is `true`.

```
template<class Rep, class Period>
  multi_lock(const chrono::duration<Rep, Period>& rel_time,
        MutexTypes&... m);
```

14 *Preconditions*: All types in `MutexTypes` meet the *Cpp17TimedLockable* requirements (32.2.5.4).

15 *Effects*: Initializes `pm` with `addressof(m)...`. Then calls `try_lock_for(rel_time)`.

16 *Postconditions*: `pm` is equal to `tuple<MutexTypes*...>{addressof(m)...}` and `owns` is `true` if `res` equals -1, otherwise `false`, where `res` is the value returned by `try_lock_for(rel_time)`.

```
template<class Clock, class Duration>
  multi_lock(const chrono::time_point<Clock, Duration>& abs_time,
        MutexTypes&... m);
```

17 *Preconditions*: All types in `MutexTypes` meet the *Cpp17TimedLockable* requirements (32.2.5.4).

18 *Effects*: Initializes `pm` with `addressof(m)...`. Then calls `try_lock_until(abs_time)`.

19 *Postconditions*: `pm` is equal to `tuple<MutexTypes*...>{addressof(m)...}` and `owns` is `true` if `res` equals -1, otherwise `false`, where `res` is the value returned by `try_lock_until(abs_time)`.

```
multi_lock(multi_lock&& u) noexcept;
```

20 *Postconditions*: `pm` is equal to `u_p.pm` and `owns` is equal to `u_p.owns` (where `u_p` is the state of `u` just prior to this construction), `u.pm` is equal to `tuple<MutexTypes*...>{}` and `u.owns` is `false`.

```
multi_lock& operator=(multi_lock&& u) noexcept;
```

21 *Effects*: Equivalent to: `multi_lock(std::move(u)).swap(*this)`.

22 *Returns*: `*this`.

```
~multi_lock();
```

23 *Effects*: If `owns` is `true`, calls `unlock()`.

32.6.X.3 Locking [thread.lock.multi.locking]

```
void lock();
```

1 *Preconditions*: If `sizeof...(MutexTypes)` does not equal 1, all types in `MutexTypes` meet the *Cpp17Lockable* requirements (32.2.5.3).

2 *Effects*: If `sizeof...(MutexTypes)` equals 0, no effects. Otherwise, if `sizeof...(MutexTypes)` equals 1, calls `get<0>(pm)->lock()`. Otherwise, calls

`lock(*get<Is>(pm)...)` where `Is` is `0, 1, ..., sizeof...(MutexTypes)-1`.

3 *Postconditions*: `owns` is `true`.

4 *Throws*: Any exception thrown by the mutex `lock()` functions. `system_error` when an exception is required (32.2.2).

5 *Error conditions*:

(5.1) — `operation_not_permitted` — if any pointer in `pm` is null.

(5.2) — `resource_deadlock_would_occur` — if on entry `owns` is `true`.

```
int try_lock();
```

6 *Preconditions*: All types in `MutexTypes` meet the *Cpp17Lockable* requirements (32.2.5.3).

7 *Effects*: If `sizeof...(MutexTypes)` equals 0, returns -1. Otherwise, if `sizeof...(MutexTypes)` equals 1, calls `get<0>(pm)->try_lock()` and returns -1 if successful, 0 otherwise. Otherwise, calls `try_lock(*get<Is>(pm)...)` where `Is` is `0, 1, ..., sizeof...(MutexTypes)-1`.

8 *Postconditions*: `owns` is `true` if the return value equals -1, otherwise `false`.

9 *Returns*: `-1` if all locks were acquired, otherwise the 0-based index of the mutex that failed to lock.

10 *Throws*: Any exception thrown by the mutex `try_lock()` functions. `system_error` when an exception is required (32.2.2).

11 *Error conditions*:

(11.1) — `operation_not_permitted` — if any pointer in `pm` is null.

(11.2) — `resource_deadlock_would_occur` — if on entry `owns` is `true`.

```
template<class Rep, class Period>
  int try_lock_for(const chrono::duration<Rep, Period>& rel_time);
```

12 *Preconditions*: All types in `MutexTypes` meet the *Cpp17TimedLockable* requirements (32.2.5.4).

13 *Effects*: Equivalent to: `return try_lock_until(chrono::steady_clock::now() + rel_time);`

```
template<class Clock, class Duration>
  int try_lock_until(const chrono::time_point<Clock, Duration>&
        abs_time);
```

14 *Preconditions*: All types in `MutexTypes` meet the *Cpp17TimedLockable* requirements (32.2.5.4).

15 *Effects*: Attempts to lock all mutexes using a deadlock-avoidance algorithm until `abs_time`. If `sizeof...(MutexTypes)` equals 0, returns -1. Otherwise, if `sizeof...(MutexTypes)` equals 1, calls `get<0>(pm)->try_lock_until(abs_time)` and returns -1 if successful, 0 otherwise. Otherwise, uses an algorithm similar to `lock()` but with timed operations respecting `abs_time`.

16 *Postconditions*: `owns` is `true` if the return value equals -1, otherwise `false`.

17 *Returns*: `-1` if all locks were acquired, otherwise the 0-based index of the mutex that failed to lock before the timeout.

18 *Throws*: Any exception thrown by the mutex `try_lock_until()` functions. `system_error` when an exception is required (32.2.2).

19 *Error conditions*:

(19.1) — `operation_not_permitted` — if any pointer in `pm` is null.

(19.2) — `resource_deadlock_would_occur` — if on entry `owns` is `true`.

```
void unlock();
```

20 *Effects*: For all *i* in `[0, sizeof...(MutexTypes))`, `get<i>(pm)->unlock()`.

21 *Postconditions*: `owns` is `false`.

22 *Throws*: `system_error` when an exception is required (32.2.2).

23 *Error conditions*:

(23.1) — `operation_not_permitted` — if on entry `owns` is `false`.

32.6.X.4 Modifiers [thread.lock.multi.mod]

```
void swap(multi_lock& u) noexcept;
```

1 *Effects*: Swaps the data members of `*this` and `u`.

```
mutex_type release() noexcept;
```

2 *Postconditions*: `pm` is equal to `tuple<MutexTypes*...>{}` and `owns` is `false`.

3 *Returns*: The previous value of `pm`.

```
template<class... MutexTypes>
  void swap(multi_lock<MutexTypes...>& x, multi_lock<MutexTypes...>& y)
       noexcept;
```

4 *Effects*: As if by `x.swap(y)`.

32.6.X.5 Observers [thread.lock.multi.obs]

```
bool owns_lock() const noexcept;
```

1 *Returns*: `owns`.

```
explicit operator bool() const noexcept;
```

2 *Returns*: `owns`.

```
mutex_type mutex() const noexcept;
```

3 *Returns*: `pm`.

10. References

- N5014: Working Draft, Standard for Programming Language C++
- P0156R2: Variadic `lock_guard` (scoped_lock)
- P3832: Timed lock for multiple mutexes